

SERVICIO NACIONAL DE APRENDIZAJE – SENA

Regional Boyacá · Centro Industrial de Mantenimiento y Manufactura – CIMM
Tecnólogo en Análisis y Desarrollo de Software – ADSO



TALLER PRÁCTICO

Introducción a Spring Boot

Desde start.spring.io hasta una aplicación web completa

Spring Web · Thymeleaf · Spring Data JPA · MySQL Driver · Spring Security

Programa:	Tecnólogo en Análisis y Desarrollo de Software – ADSO
Centro:	CIMM – Centro Industrial de Mantenimiento y Manufactura
Competencia:	225501003 – Desarrollar e integrar componentes y servicios
Requisito previo:	Java Servlets + JSP + MySQL + Maven
Spring Boot:	3.3.x Java 17 Maven
Duración estimada:	16 horas — 4 sesiones de laboratorio
Versión:	1.0 – 2026

1. ¿Por qué Spring Boot después de Servlets?

Ya sabes construir aplicaciones web Java con Servlets, JSP y MySQL. Esa arquitectura te enseñó los fundamentos reales: el protocolo HTTP, el ciclo request/response, el patrón DAO, las sesiones. Ahora das el siguiente paso profesional.

De Servlets a Spring Boot — qué cambia y qué permanece igual

LO QUE PERMANECE IGUAL:

- Java sigue siendo el lenguaje — mismas clases, misma lógica de negocio
- El protocolo HTTP es el mismo — GET, POST, parámetros, sesiones
- MySQL y el patrón DAO siguen siendo válidos
- La arquitectura en capas (controlador → servicio → datos) es la misma idea

LO QUE CAMBIA (para mejor):

- No más web.xml — Spring Boot lo configura todo automáticamente
- No más Tomcat externo — Spring Boot lleva Tomcat EMBEBIDO en el JAR
- No más PreparedStatement manual — Spring Data JPA lo gestiona por ti
- Seguridad, validación e i18n integrados como dependencias Maven
- mvn spring-boot:run en lugar de desplegar un WAR en Tomcat

1.1 Equivalencias clave: Servlets → Spring Boot

Concepto	En Java Servlets	En Spring Boot
Controlador HTTP	@WebServlet + doGet/doPost	@Controller + @GetMapping/@PostMapping
Vista	login.jsp con JSTL	login.html con Thymeleaf (th:text, th:each)
Acceso a BD	PacienteDAO con PreparedStatement	PacienteRepository (Spring Data JPA)
Entidad / Modelo	Paciente.java (POJO manual)	@Entity Paciente.java (JPA gestiona el SQL)
Configuración BD	Conexion.java con URL hardcodeada	application.properties con spring.datasource.*
Servidor	Tomcat 9 externo (WAR)	Tomcat embebido (JAR ejecutable)
Arrancar la app	Desplegar WAR en Tomcat	mvn spring-boot:run o java -jar app.jar

Concepto	En Java Servlets	En Spring Boot
Filtro de seguridad	AuthFilter.java (@WebFilter)	Spring Security (configuración en Java)
Internacionalización	LocaleFilter + messages.properties	MessageSource configurado automáticamente
Inyección de dependencias	new PacienteDAO() manual	@Autowired / constructor injection

2. Crear el Proyecto en start.spring.io

Spring Initializr es el generador oficial de proyectos Spring Boot. En segundos genera la estructura Maven completa con las dependencias seleccionadas. Accede a <https://start.spring.io> desde el navegador.

2.1 Parámetros del proyecto

Campo	Valor para este taller
Project	Maven
Language	Java
Spring Boot	3.3.x (la más reciente estable — no usar SNAPSHOT)
Group	co.sena.cimm.adso
Artifact	vacunacion-spring
Name	vacunacion-spring (se llena automático)
Description	Sistema de Gestión de Vacunación — SENA C IMM
Package name	co.sena.cimm.adso.vacunacion (se llena automático)
Packaging	Jar (Tomcat embebido — diferente al WAR de Servlets)
Java	17

2.2 Dependencias — El núcleo del taller

Haz clic en 'ADD DEPENDENCIES' y agrega estas 5 dependencias. Cada una reemplaza o mejora algo que hacías manualmente en el proyecto de Servlets:

Dependencia	Buscarla como...	Qué reemplaza del proyecto de Servlets
Spring Web	Spring Web	La Servlet API + el mapeo de rutas en web.xml + @WebServlet
Thymeleaf	Thymeleaf	Los archivos .jsp con etiquetas JSTL <c:forEach>, <fmt:message>

Dependencia	Buscarla como...	Qué reemplaza del proyecto de Servlets
Spring Data JPA	Spring Data JPA	El PacienteDAO con PreparedStatement, ResultSet y mapeo manual
MySQL Driver	MySQL Driver	El mysql-connector-java en el pom.xml (mismo driver, diferente configuración)
Spring Boot DevTools	Spring Boot DevTools	El reinicio manual de Tomcat — DevTools recarga la app automáticamente

Haz clic en GENERATE → descarga vacunacion-spring.zip → descomprime → importa en IntelliJ IDEA o NetBeans.

¿Qué genera start.spring.io exactamente?	
Al hacer GENERATE obtienes un ZIP con esta estructura lista para usar:	
vacunacion-spring/	
— pom.xml	← dependencias ya configuradas
— mvnw / mvnw.cmd	← Maven Wrapper (no necesitas mvn instalado)
— src/	
— main/	
— java/co/sena/cimm/adso/vacunacion/	
— VacunacionSpringApplication.java	← punto de entrada (main())
— resources/	
— application.properties	← configuración BD, servidor, i18n
— static/	← CSS, JS, imágenes
— templates/	← archivos .html con Thymeleaf
— test/	
— java/...	← pruebas JUnit 5 (ya vienen configuradas)

3. El pom.xml Generado — Entender cada pieza

Spring Initializr genera el pom.xml completo. Es importante entender qué hace cada sección, especialmente las diferencias con el pom.xml del proyecto de Servlets:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <!-- ① PARENT — el corazón de Spring Boot -->
  <!-- Gestiona las versiones de TODAS las dependencias automáticamente -->
  <!-- Ya no necesitas escribir <version> en cada dependencia -->
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
  </parent>

  <groupId>co.sena.cimm.adso</groupId>
  <artifactId>vacunacion-spring</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging> <!-- JAR con Tomcat embebido, no WAR -->

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>

    <!-- ② Spring Web — controladores HTTP (@Controller, @GetMapping) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <!-- Sin <version>: el parent la gestiona automáticamente -->
    </dependency>

    <!-- ③ Thymeleaf — motor de plantillas HTML (reemplaza JSP+JSTL) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>

    <!-- ④ Spring Data JPA — acceso a BD sin SQL manual -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- ⑤ MySQL Driver — el mismo de siempre -->
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
    </dependency>

    <!-- ⑥ DevTools — recarga automática al guardar (solo desarrollo) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>

    <!-- ⑦ JUnit 5 - ya viene incluido automáticamente -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

</dependencies>

<build>
    <plugins>
        <!-- Plugin que crea el JAR ejecutable con Tomcat embebido -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

4. application.properties — La Configuración Central

En el proyecto de Servlets la configuración estaba dispersa en Conexion.java, web.xml y LocaleFilter.java. En Spring Boot todo va en un solo archivo: src/main/resources/application.properties. Abre ese archivo y reemplaza su contenido con lo siguiente:

```

# =====
# Sistema de Gestión de Vacunación — SENA CIMM
# Configuración completa de Spring Boot
# =====

# — Servidor —
spring.application.name=vacunacion-spring
server.port=8080
# Context path: la app responde en http://localhost:8080/
# (diferente al WAR que respondía en /captcha/)

# — Base de datos MySQL —
spring.datasource.url=jdbc:mysql://localhost:3306/vacunacion_db?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# — JPA / Hibernate —
# update: Hibernate adapta las tablas si cambia el modelo (desarrollo)
# validate: solo verifica que las tablas existen (producción)
spring.jpa.hibernate.ddl-auto=update
# Muestra el SQL generado por JPA en la consola (útil para aprender)
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect

```

```
# — Thymeleaf —————
# Deshabilitar caché en desarrollo (recarga plantillas sin reiniciar)
spring.thymeleaf.cache=false
spring.thymeleaf.encoding=UTF-8

# — Internacionalización —————
# Mismo concepto que messages.properties del proyecto de Servlets
spring.messages.basename=messages
spring.messages.encoding=UTF-8

# — Logging —————
logging.level.co.sena.cimm.adso=DEBUG
logging.level.org.hibernate.SQL=DEBUG
```

5. La Clase Principal — VacunacionSpringApplication.java

Spring Boot ya generó esta clase automáticamente. Es el reemplazo de web.xml — no necesitas configurar servlets, filtros ni listeners manualmente. Una sola anotación arranca todo:

```
package co.sena.cimm.adso.vacunacion;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * Punto de entrada de la aplicación Spring Boot.
 *
 * @SpringBootApplication equivale a tres anotaciones en una:
 *   @Configuration → esta clase puede definir beans de Spring
 *   @EnableAutoConfiguration → Spring configura automáticamente todo lo que
 *                               detecte en el classpath (Thymeleaf, JPA, etc.)
 *   @ComponentScan → busca @Controller, @Service, @Repository, etc.
 *                   en este paquete y subpaquetes automáticamente
 */
@SpringBootApplication
public class VacunacionSpringApplication {

    public static void main(String[] args) {
        // Arranca Tomcat embebido, carga el contexto de Spring y lanza la app
        SpringApplication.run(VacunacionSpringApplication.class, args);
        // Equivale a: arrancar Tomcat + desplegar el WAR, en una sola línea
    }
}
```

5.1 Cómo arrancar la aplicación

Método	Comando / Acción
Terminal Maven	mvn spring-boot:run (desde la carpeta raíz del proyecto)
IntelliJ IDEA	Clic derecho en VacunacionSpringApplication → Run
NetBeans	Clic derecho en el proyecto → Run Maven → spring-boot:run


```

private String apellidos;

// unique=true equivale a UNIQUE en el DDL
@Column(nullable = false, unique = true, length = 20)
private String documento;

@Column(name = "fecha_nacimiento", nullable = false)
private LocalDate fechaNacimiento;

// Constructor vacío - requerido por JPA
public Paciente() {}

// Constructor de conveniencia
public Paciente(String nombres, String apellidos,
                String documento, LocalDate fechaNacimiento) {
    this.nombres      = nombres;
    this.apellidos     = apellidos;
    this.documento     = documento;
    this.fechaNacimiento = fechaNacimiento;
}

/** Método de negocio - igual que en el proyecto de Servlets */
public String getNombreCompleto() {
    return nombres + " " + apellidos;
}

// Getters y Setters
public Long getId() { return id; }
public String getNombres() { return nombres; }
public void setNombres(String n) { this.nombres = n; }
public String getApellidos() { return apellidos; }
public void setApellidos(String a) { this.apellidos = a; }
public String getDocumento() { return documento; }
public void setDocumento(String d) { this.documento = d; }
public LocalDate getFechaNacimiento() { return fechaNacimiento; }
public void setFechaNacimiento(LocalDate f) { this.fechaNacimiento = f; }
}

```

6.2 Vacuna.java — Entidad JPA

```

package co.sena.cimm.adso.vacunacion.model;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "vacunas",
        uniqueConstraints = @UniqueConstraint(columnNames = {"lote", "nombre"}))
public class Vacuna {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;
}

```

```

@Column(nullable = false, length = 50)
private String lote;

@Column(nullable = false, length = 100)
private String laboratorio;

@Column(name = "fecha_vencimiento", nullable = false)
private LocalDate fechaVencimiento;

public Vacuna() {}

public Vacuna(String nombre, String lote,
               String laboratorio, LocalDate fechaVencimiento) {
    this.nombre = nombre;
    this.lote = lote;
    this.laboratorio = laboratorio;
    this.fechaVencimiento = fechaVencimiento;
}

public boolean estaVencida() {
    return LocalDate.now().isAfter(fechaVencimiento);
}

// Getters y Setters
public Long getId() { return id; }
public String getNombre() { return nombre; }
public void setNombre(String n) { this.nombre = n; }
public String getLote() { return lote; }
public void setLote(String l) { this.lote = l; }
public String getLaboratorio() { return laboratorio; }
public void setLaboratorio(String l){ this.laboratorio = l; }
public LocalDate getFechaVencimiento(){ return fechaVencimiento; }
public void setFechaVencimiento(LocalDate f){ this.fechaVencimiento = f; }
}
}

```

7. Repositorios — Spring Data JPA reemplaza el DAO

En el proyecto de Servlets escribías `PacienteDAO.java` con 80+ líneas de `PreparedStatement`, `ResultSet` y `try/catch`. Spring Data JPA genera todo ese código automáticamente. Solo defines una interfaz:

7.1 `PacienteRepository.java`

```

package co.sena.cimm.adso.vacunacion.repository;

import co.sena.cimm.adso.vacunacion.model.Paciente;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;
import java.util.List;

/**
 * Solo la interfaz — Spring genera la implementación completa automáticamente.
 * JpaRepository<Paciente, Long> ya incluye sin escribir código:

```

```

*   save(p)           → INSERT o UPDATE según si el id es null o no
*   findById(id)      → SELECT * FROM pacientes WHERE id = ?
*   findAll()         → SELECT * FROM pacientes
*   deleteById(id)   → DELETE FROM pacientes WHERE id = ?
*   count()           → SELECT COUNT(*) FROM pacientes
*   existsById(id)    → SELECT EXISTS(SELECT 1 FROM pacientes WHERE id = ?)
*/
@Repository
public interface PacienteRepository extends JpaRepository<Paciente, Long> {

    // Spring genera el SQL a partir del nombre del método:
    // findByIdDocumento → SELECT * FROM pacientes WHERE documento = ?
    Optional<Paciente> findByIdDocumento(String documento);

    // findByIdNombresContainingIgnoreCase →
    //   SELECT * FROM pacientes WHERE UPPER(nombres) LIKE UPPER('%texto%')
    List<Paciente>
    findByIdNombresContainingIgnoreCaseOrApellidosContainingIgnoreCase(
        String nombres, String apellidos
    );

    // Método con @Query para SQL personalizado (cuando el nombre sería muy largo)
    @org.springframework.data.jpa.repository.Query(
        "SELECT p FROM Paciente p WHERE p.documento = :doc"
    )
    Optional<Paciente> buscarPorDocumento(
        @org.springframework.data.repository.query.Param("doc") String doc
    );
}

```

7.2 VacunaRepository.java

```

package co.sena.cimm.adso.vacunacion.repository;

import co.sena.cimm.adso.vacunacion.model.Vacuna;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

@Repository
public interface VacunaRepository extends JpaRepository<Vacuna, Long> {

    Optional<Vacuna> findByIdAndNombre(String lote, String nombre);

    // Vacunas cuya fecha_vencimiento es posterior a hoy (no vencidas)
    List<Vacuna> findByFechaVencimientoAfter(LocalDate fecha);

    List<Vacuna> findByLaboratorio(String laboratorio);
}

```

8. Capa de Servicio — La Lógica de Negocio

En el proyecto de Servlets la lógica de negocio estaba mezclada en el DAO o en el Servlet. En Spring Boot se extrae a una clase `@Service`, que es el intermediario entre el controlador y el repositorio:

```
package co.sena.cimm.adso.vacunacion.service;

import co.sena.cimm.adso.vacunacion.model.Paciente;
import co.sena.cimm.adso.vacunacion.repository.PacienteRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.Optional;

/**
 * @Service indica a Spring que esta clase contiene lógica de negocio.
 * Spring la registra como un bean y la inyecta donde se necesite.
 */
@Service
@Transactional // cada método corre dentro de una transacción de BD
public class PacienteService {

    // Inyección de dependencias por constructor — la forma recomendada
    // Spring inyecta automáticamente la implementación de PacienteRepository
    private final PacienteRepository pacienteRepository;

    public PacienteService(PacienteRepository pacienteRepository) {
        this.pacienteRepository = pacienteRepository;
    }

    /** Lista todos los pacientes ordenados por apellido */
    @Transactional(readOnly = true)
    public List<Paciente> listarTodos() {
        return pacienteRepository.findAll()
            .stream()
            .sorted((a, b) -> a.getApellidos().compareTo(b.getApellidos()))
            .toList();
    }

    /** Busca un paciente por ID. Lanza excepción si no existe */
    @Transactional(readOnly = true)
    public Paciente buscarPorId(Long id) {
        return pacienteRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Paciente con ID " + id + " no encontrado"));
    }

    /** Busca por número de documento */
    @Transactional(readOnly = true)
    public Optional<Paciente> buscarPorDocumento(String documento) {
        return pacienteRepository.findByDocumento(documento);
    }

    /** Guarda o actualiza un paciente */
    public Paciente guardar(Paciente paciente) {
        return pacienteRepository.save(paciente);
    }
}
```

```
/** Elimina un paciente por ID */  
public void eliminar(Long id) {  
    if (!pacienteRepository.existsById(id))  
        throw new RuntimeException("Paciente no encontrado");  
    pacienteRepository.deleteById(id);  
}  
}
```

9. Controladores — Reemplazo del Servlet

En el proyecto de Servlets tenías `PacienteServlet.java` con `doGet()` y `doPost()` y un switch para enrutar las acciones. En Spring Boot usas `@Controller` con métodos anotados por ruta y método HTTP:

9.1 PacienteController.java — CRUD completo

```
package co.sena.cimm.adso.vacunacion.controller;

import co.sena.cimm.adso.vacunacion.model.Paciente;
import co.sena.cimm.adso.vacunacion.service.PacienteService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

/**
 * @Controller — equivale a @WebServlet del proyecto de Servlets
 * @RequestMapping("/pacientes") — prefijo base de todas las rutas
 */
@Controller
@RequestMapping("/pacientes")
public class PacienteController {

    private final PacienteService pacienteService;

    public PacienteController(PacienteService pacienteService) {
        this.pacienteService = pacienteService;
    }

    // — GET /pacientes → listar todos —————
    // Equivale a: case "listar": en el doGet() del PacienteServlet
    @GetMapping
    public String listar(Model model) {
        // Model es el equivalente de request.setAttribute() en Servlets
        model.addAttribute("pacientes", pacienteService.listarTodos());
        return "pacientes/lista"; // → templates/pacientes/lista.html
    }

    // — GET /pacientes/nuevo → mostrar formulario vacío —————
    @GetMapping("/nuevo")
    public String mostrarFormularioNuevo(Model model) {
        model.addAttribute("paciente", new Paciente());
        return "pacientes/formulario"; // → templates/pacientes/formulario.html
    }

    // — POST /pacientes → guardar nuevo paciente —————
    // @ModelAttribute enlaza automáticamente los campos del form a Paciente
    // Equivale a: request.getParameter("nombres") + setNombres(...) manual
    @PostMapping
    public String guardar(@ModelAttribute Paciente paciente,
                          RedirectAttributes redirectAttr) {
        pacienteService.guardar(paciente);
        // Patrón POST/Redirect/GET — igual que en el proyecto de Servlets
        redirectAttr.addFlashAttribute("mensaje", "Paciente guardado
correctamente");
        return "redirect:/pacientes";
    }
}
```

```

// — GET /pacientes/{id}/editar → formulario con datos del paciente —
@GetMapping("/{id}/editar")
public String mostrarFormularioEditar(@PathVariable Long id, Model model) {
    model.addAttribute("paciente", pacienteService.buscarPorId(id));
    return "pacientes/formulario";
}

// — POST /pacientes/{id} → actualizar paciente —————
@PostMapping("/{id}")
public String actualizar(@PathVariable Long id,
                        @ModelAttribute Paciente paciente,
                        RedirectAttributes redirectAttr) {
    // El ID del path se asigna al objeto para que JPA haga UPDATE
    paciente.setId(id); // necesitamos agregar setId() a la entidad
    pacienteService.guardar(paciente);
    redirectAttr.addFlashAttribute("mensaje", "Paciente actualizado");
    return "redirect:/pacientes";
}

// — GET /pacientes/{id}/eliminar → eliminar paciente —————
@GetMapping("/{id}/eliminar")
public String eliminar(@PathVariable Long id,
                      RedirectAttributes redirectAttr) {
    pacienteService.eliminar(id);
    redirectAttr.addFlashAttribute("mensaje", "Paciente eliminado");
    return "redirect:/pacientes";
}
}

```

10. Vistas Thymeleaf — Reemplazo de JSP

Las vistas .html de Thymeleaf reemplazan los archivos .jsp. Son HTML válido que funciona directamente en el navegador (sin servidor). Las expresiones Thymeleaf (th:*) se evalúan cuando Spring las procesa:

10.1 Comparación JSP vs Thymeleaf

Función	JSP (proyecto Servlets)	Thymeleaf (Spring Boot)
Mostrar variable	<code>\${paciente.nombres}</code>	<code>th:text="\${paciente.nombres}"</code>
Iterar lista	<code><c:forEach var='p' items='\${lista}'></code>	<code><tr th:each='p : \${pacientes}'></code>
Condición	<code><c:if test='\${condicion}'></code>	<code>th:if="\${condicion}"</code>
URL del formulario	<code>action='Servlet?accion=guardar'</code>	<code>th:action="@{/pacientes}"</code>
Valor de campo	<code>value='\${paciente.nombres}'</code>	<code>th:value="\${paciente.nombres}"</code>
Enlace / href	<code>href='/captcha/pacientes'</code>	<code>th:href="@{/pacientes}"</code>
Mensajes i18n	<code><fmt:message key='titulo'/></code>	<code>th:text="#{titulo}"</code>
Incluir fragmento	<code><%@ include file='header.jsp'%></code>	<code>th:replace=~{fragments/header :: nav}'</code>

10.2 templates/pacientes/lista.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Pacientes – Sistema de Vacunación</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
<div class="container mt-4">

  <!-- Encabezado -->
  <div class="d-flex justify-content-between align-items-center mb-3">
    <h2 class="text-success">Pacientes</h2>
    <!-- th:href genera la URL correcta con el context path -->
    <a th:href="@{/pacientes/nuevo}" class="btn btn-success">
      + Nuevo Paciente
    </a>
  </div>

  <!-- Mensaje de éxito (después de guardar/eliminar) -->
  <!-- th:if evalúa la condición – equivale a <c:if> en JSP -->
  <div th:if="${mensaje}" class="alert alert-success"
th:text="${mensaje}"></div>

  <!-- Tabla de pacientes -->
  <table class="table table-striped table-hover bg-white shadow-sm">
    <thead class="table-success">
      <tr>
        <th>ID</th>
        <th>Nombre Completo</th>
        <th>Documento</th>
        <th>Fecha Nacimiento</th>
        <th>Acciones</th>
      </tr>
    </thead>
    <tbody>
      <!-- th:each itera la lista – equivale a <c:forEach> en JSP -->
      <tr th:each="paciente : ${pacientes}">
        <td th:text="${paciente.id}"></td>
        <!-- Llama al método getNombreCompleto() del modelo -->
        <td th:text="${paciente.nombreCompleto}"></td>
        <td th:text="${paciente.documento}"></td>
        <td th:text="${paciente.fechaNacimiento}"></td>
        <td>
          <!-- @{/pacientes/{id}/editar(id=${paciente.id})} -->
          <a th:href="@{/pacientes/{id}/editar(id=${paciente.id})}"
class="btn btn-sm btn-primary">Editar</a>
          <a th:href="@{/pacientes/{id}/eliminar(id=${paciente.id})}"
class="btn btn-sm btn-danger"
onclick="return confirm('¿Eliminar paciente?')">
            Eliminar
          </a>
        </td>
      </tr>
      <!-- Mensaje cuando no hay pacientes -->
      <tr th:if="${#lists.isEmpty(pacientes)}">
```



```

        </div>
        <div class="mb-3">
            <label class="form-label">Fecha de Nacimiento</label>
            <input type="date" th:field="*{fechaNacimiento}"
                class="form-control" required>
        </div>
        <div class="d-flex gap-2">
            <button type="submit" class="btn btn-success">Guardar</button>
            <a th:href="@{/pacientes}" class="btn btn-
secondary">Cancelar</a>
        </div>
    </form>
</div>
</div>
</div>
</body>
</html>

```

11. Estructura Final del Proyecto

```

vacunacion-spring/src/main/
├── java/co/sena/cimm/adso/vacunacion/
│   ├── VacunacionSpringApplication.java ← punto de entrada
│   │   (@SpringBootApplication)
│   ├── model/
│   │   ├── Paciente.java ← @Entity
│   │   ├── Vacuna.java ← @Entity
│   │   └── Usuario.java ← @Entity (para el taller - a crear)
│   ├── repository/
│   │   ├── PacienteRepository.java ← extends JpaRepository
│   │   └── VacunaRepository.java ← extends JpaRepository
│   ├── service/
│   │   ├── PacienteService.java ← @Service @Transactional
│   │   └── VacunaService.java ← @Service (a crear en ejercicios)
│   └── controller/
│       ├── PacienteController.java ← @Controller @RequestMapping
│       ├── VacunaController.java ← (a crear en ejercicios)
│       └── HomeController.java ← redirige / → /pacientes
└── resources/
    ├── application.properties ← BD, JPA, Thymeleaf, i18n
    ├── messages.properties ← textos en español (i18n)
    ├── messages_en.properties ← textos en inglés
    ├── static/
    │   └── css/vacunacion.css ← estilos personalizados
    └── templates/
        ├── index.html ← página de bienvenida
        ├── pacientes/
        │   ├── lista.html
        │   └── formulario.html
        └── vacunas/
            ├── lista.html ← (a crear en ejercicios)
            └── formulario.html ← (a crear en ejercicios)

```

12. Ejercicios Prácticos por Sesión

Distribución — 4 sesiones de laboratorio

Sesión 1 (4 h): start.spring.io, pom.xml, application.properties,

clase principal, primera ejecución.

Sesión 2 (4 h): Entidades JPA, repositorios, primer CRUD de Pacientes.

Sesión 3 (4 h): Capa de servicio, controlador completo, vistas Thymeleaf.

Sesión 4 (4 h): Módulo de Vacunas completo + HomeController + estilo CSS.

12.1 Sesión 1 — Crear y Arrancar el Proyecto

#	Actividad
1	Abre https://start.spring.io y configura el proyecto con los parámetros de la sección 2. Descarga el ZIP, descomprime e importa en tu IDE.
2	Abre pom.xml y verifica que las 5 dependencias estén presentes (sección 3). Observa la diferencia con el pom.xml del proyecto de Servlets: no hay versiones individuales.
3	Abre application.properties y reemplaza su contenido con la configuración de la sección 4. Ajusta el usuario y contraseña de MySQL según tu instalación local.
4	Ejecuta <code>mvn spring-boot:run</code> . Verifica que la consola muestra 'Started VacunacionSpringApplication'. Abre http://localhost:8080 — debe aparecer un error 404 (normal, aún no hay controlador).
5	Crea HomeController.java en el paquete controller con <code>@GetMapping("/")</code> que retorne "redirect:/pacientes". Crea un templates/index.html mínimo.
6	Compara el arranque de Spring Boot con el arranque de Tomcat+WAR: ¿cuántos pasos tiene cada uno? Anota en tu bitácora las diferencias.
7	RETO: Cambia <code>server.port=9090</code> en application.properties, reinicia y verifica que la app responde en http://localhost:9090 . Luego vuelve a 8080.

12.2 Sesión 2 — Entidades JPA y Repositorios

#	Actividad
1	Crea Paciente.java con las anotaciones JPA (sección 6.1). Arranca la app — Hibernate debe crear o actualizar la tabla 'pacientes' en MySQL automáticamente (verifica con DBeaver o MySQL Workbench).
2	Crea PacienteRepository.java (sección 7.1). Observa que solo es una interfaz — Spring genera la implementación. Compara con el PacienteDAO.java del proyecto de Servlets (80+ líneas vs 10 líneas).
3	Crea Vacuna.java con <code>@Entity</code> y sus anotaciones. Arranca — Hibernate crea la tabla 'vacunas'. Verifica el UNIQUE KEY <code>uk_lote_nombre</code> en MySQL.

#	Actividad
4	Crea VacunaRepository.java con los métodos findByLoteAndNombre() y findByFechaVencimientoAfter().
5	ANÁLISIS: Cambia spring.jpa.show-sql=true y arranca. Inserta un paciente manualmente en MySQL. ¿Qué SQL genera Hibernate cuando llamas a findAll()? ¿Coincide con el SQL que escribías manualmente?
6	RETO: Agrega un campo @Column(name='telefono') a Paciente.java. Reinicia — Hibernate debe agregar la columna a la tabla automáticamente (ddl-auto=update). Verifica en MySQL.

12.3 Sesión 3 — Servicio, Controlador y Vistas

#	Actividad
1	Crea PacienteService.java (sección 8). Inyecta PacienteRepository por constructor. Implementa listarTodos(), buscarPorId(), guardar() y eliminar().
2	Crea PacienteController.java (sección 9). Verifica que el IDE detecta la inyección automática de PacienteService sin new.
3	Crea templates/pacientes/lista.html (sección 10.2). Accede a http://localhost:8080/pacientes — debe aparecer la tabla (vacía). Inserta un paciente manualmente en MySQL y recarga.
4	Crea templates/pacientes/formulario.html (sección 10.3). Prueba el flujo completo: Nuevo Paciente → llenar formulario → Guardar → verificar en la lista.
5	Prueba Editar: clic en Editar para un paciente → modificar el nombre → Guardar → verificar el cambio en la lista.
6	Prueba Eliminar: clic en Eliminar → confirmar → verificar que desaparece. ¿El mensaje de confirmación funciona con el onclick de JS?
7	ANÁLISIS: Compara PacienteController (Spring Boot) con PacienteServlet (Servlets). ¿Cuántas líneas tiene cada uno? ¿Cuál es más legible? ¿Por qué?

12.4 Sesión 4 — Módulo de Vacunas + Estilo

#	Actividad
1	Crea VacunaService.java con los métodos listarTodas(), listarDisponibles() (fecha_vencimiento > hoy), guardar() y eliminar().
2	Crea VacunaController.java con @RequestMapping("/vacunas") y los mismos métodos GET/POST del PacienteController pero para Vacuna.
3	Crea templates/vacunas/lista.html y templates/vacunas/formulario.html. En la lista agrega un badge de color: 'Vencida' en rojo si estaVencida() es true.
4	Crea static/css/vacunacion.css con los colores corporativos del sistema (verde SENA #39A900, azul #1E4D78). Enlázalo en ambas vistas con <link th:href="@{/css/vacunacion.css}">.
5	Agrega un navbar compartido: crea templates/fragments/navbar.html con th:fragment='nav'. Inclúyelo en todas las páginas con th:replace=~{fragments/navbar :: nav}'.

#	Actividad
6	RETO: Agrega i18n en Spring Boot. Crea messages.properties con app.titulo=Sistema de Vacunación y messages_en.properties con app.titulo=Vaccination System. Configura LocaleChangeInterceptor para cambiar con ?lang=en.
7	PRESENTACIÓN: Demuestra el CRUD completo de Pacientes y Vacunas al instructor. Compara con el sistema de Servlets original — ¿qué es más simple de mantener?

13. Preguntas de Reflexión



Responde en tu bitácora — mínimo 5 líneas por pregunta

Estas preguntas se socializan en plenaria al cierre de la sesión 4.

13.1 Sobre la Arquitectura

- En el proyecto de Servlets tenías: Paciente.java + PacienteDAO.java + PacienteServlet.java. En Spring Boot tienes: Paciente.java (@Entity) + PacienteRepository.java + PacienteService.java + PacienteController.java. ¿Hay más archivos — por qué esto es mejor y no peor?
- Spring Boot usa inyección de dependencias: el controlador recibe el servicio sin hacer new PacienteService(). En el proyecto de Servlets hacías PacienteDAO dao = new PacienteDAO(). ¿Cuál es la ventaja de no crear los objetos manualmente? ¿Qué problema resuelve?
- La anotación @Transactional en PacienteService garantiza que si falla una operación en la BD, se hace rollback automático. En el proyecto de Servlets, ¿tenías transacciones? ¿Qué pasaba si el INSERT de un registro fallaba a mitad?

13.2 Sobre JPA y Thymeleaf

- En PacienteRepository defines el método findByDocumento(String documento) — Spring genera el SQL automáticamente. ¿Cómo sabe Spring qué SQL generar solo con el nombre del método? ¿Cuál sería el SQL para findByNombresContainingIgnoreCase(String texto)?
- th:field='*{nombres}' en Thymeleaf hace tres cosas a la vez: establece name, id y value del campo HTML. ¿Cuántas líneas de JSP/JSTL necesitabas para lograr lo mismo en el proyecto de Servlets? Escribe el equivalente JSP.

13.3 Sobre el Ecosistema Spring

- spring.jpa.hibernate.ddl-auto=update permite que Hibernate modifique las tablas automáticamente al cambiar el modelo. ¿Por qué en producción se usa ddl-auto=validate en lugar de update? ¿Qué riesgo tiene usar update en un sistema con datos reales?
- DevTools recarga la aplicación automáticamente al guardar archivos. ¿Cómo se compara con el flujo de desarrollo en Servlets (guardar → compilar → desplegar WAR → reiniciar Tomcat)? ¿Cuánto tiempo ahorras por hora de desarrollo?

14. Errores Frecuentes y Soluciones

Error / Síntoma	Causa y Solución
Failed to configure a DataSource	application.properties no tiene spring.datasource.url configurado, o la BD MySQL no existe. Crear la BD vacunacion_db y ajustar las credenciales.
No qualifying bean of type PacienteService	Olvidaste @Service en PacienteService.java o @Controller en el controlador. Spring solo inyecta beans anotados.
Whitelabel Error Page 404	La ruta del @GetMapping no coincide con la URL del navegador, o el método retorna el nombre de una vista que no existe en templates/.
TemplateInputException: template not found	El método del controlador retorna 'pacientes/lista' pero el archivo está en templates/pacientes/lista.html — verificar ruta y nombre exactos.
Could not write JSON: infinite recursion	Relación @ManyToOne/@OneToMany circular sin @JsonIgnore. Solución con Thymeleaf: no hay problema porque no serializa JSON.
Tabla no se crea automáticamente en MySQL	spring.jpa.hibernate.ddl-auto no está configurado. Agregar ddl-auto=update en application.properties.
th:field no funciona en el formulario	Falta th:object='\${paciente}' en la etiqueta <form>. th:field requiere que el objeto esté enlazado al formulario.
Property or field 'id' cannot be found (Thymeleaf)	La entidad no tiene getter getId(). Verificar que todos los getters estén definidos en la entidad @Entity.
Circular dependency between PacienteService y otro	Dos @Service se inyectan mutuamente. Solución: refactorizar la lógica a un tercer servicio o usar @Lazy.

15. Criterios de Evaluación

#	Criterio de Desempeño	Puntos	Instrumento
1	Proyecto generado correctamente en start.spring.io y ejecutando en localhost:8080	10 %	Demostración
2	application.properties configurado correctamente — Hibernate crea las tablas en MySQL	10 %	Revisión BD
3	Paciente.java con anotaciones JPA correctas (@Entity, @Id, @Column)	10 %	Revisión código
4	PacienteRepository con mínimo 2 métodos de búsqueda personalizados	10 %	Revisión código
5	PacienteService con @Service, @Transactional e inyección por constructor	15 %	Revisión código
6	PacienteController con CRUD completo (5 rutas) — funcional en el navegador	20 %	Demostración
7	Vistas Thymeleaf: lista con th:each + formulario con th:field — sin texto hardcodeado en Java	15 %	Revisión código

#	Criterio de Desempeño	Puntos	Instrumento
8	Preguntas de reflexión respondidas en la bitácora	10 %	Lista de chequeo
BONUS A	Módulo de Vacunas completo (CRUD + badge de vencimiento)	+0,3	Demostración
BONUS B	Navbar compartido con th:replace y i18n con ?lang=en	+0,2	Demostración

Condiciones de no aprobación automática

- Usar @Autowired en campo (field injection) en lugar de inyección por constructor.
- SQL escrito manualmente en el repositorio cuando Spring Data puede generarlo.
- Lógica de negocio (reglas, cálculos, validaciones) dentro del controlador.
- La app no arranca con mvn spring-boot:run sin modificaciones manuales.
- Credenciales de BD hardcodeadas en el código Java (no en application.properties).

SENA · Servicio Nacional de Aprendizaje

Centro Industrial de Mantenimiento y Manufactura – CIMM | Regional Boyacá
Tecnólogo en Análisis y Desarrollo de Software – ADSO | 2026